Database Systems (CS F212)

First Semester 2026-27

A Project Report on

Design and Implementation of a Database-Backed Music Streaming Platform

Mid-Semester Submission

Database Design and Application Design

Submitted by

Ritwik Sethi	2023B1A71385H
Arihant Rakhecha	2023B1A70998H
Shreyas Ghosh	2024A1PS0826H
Shreya Agrawal	2023B5A30794H
Piyush Gaikwad	2024A1PS0859H

Birla Institute of Technology and Science, Pilani

Hyderabad Campus

October 2026

PART A: DATABASE DESIGN DOCUMENT

1 Requirements Analysis

1.1 Problem and users

We are building the database behind a music streaming platform. The platform has to keep track of a music catalogue (artists, their albums, the songs on those albums and who performed on each song) and of everything users do with it: the playlists they build, the songs they like, every song they play, and their subscription. Because almost every piece of data points at some other piece, we want the database itself to guarantee consistency rather than leave it to application code. The same data also has to answer analytical questions, such as which songs are trending in a country or what a user listened to most this year.

The system has three kinds of users. A *listener* searches the catalogue, plays and likes songs, builds playlists, manages a subscription and looks at charts and a personal "Wrapped" summary. An *artist* is a user account tied to one artist profile; through it the artist releases albums and sees statistics for their own songs. An *admin* maintains users and the catalogue and views platform-wide reports. Actual audio streaming, real payment processing and recommendations are outside the scope of this project.

1.2 Assumptions

1. Every album is released by exactly one artist. Other artists who appear on a track are credited per song, through *Performs*.
2. A single is treated as an album whose type is `'single'`, so a song's release date is simply the release date of its album.
3. A user has at most one subscription record. A user without one is on the free tier.
4. Each artist has one genre. An artist profile is managed by at most one user account, and a user manages at most one profile.
5. One listening event is identified by who played it, which song it was and when it was played.
6. Users must be at least 13 years old, and free users can own at most five playlists.

1.3 Business rules

ID	Rule	How it is enforced
BR1	A user cannot like the same song twice, and a playlist cannot hold the same song twice	Composite primary key
BR2	Every album has a releasing artist, every song an album and every playlist an owner	NOT NULL foreign key
BR3	Every song has at least one performer with the role `'Lead'`	`release_album()` transaction
BR4	A song cannot be played before its album was released, or at a future time	Trigger T1
BR5	A song cannot be added to a playlist before that playlist existed	Trigger T2
BR6	A song's play count always equals the number of times it has been played	Trigger T3
BR7	Users are at least 13 years old, and a subscription cannot start in the future	Trigger T4
BR8	A user without an active paid subscription owns at most five playlists	Trigger T5
BR9	An account is of type `'artist'` exactly when it is linked to an artist profile	CHECK
BR10	A subscription ends after it starts; status values and email addresses are well formed	CHECK

2 ER Model

The model has eight entity types. Seven are strong entities with their own keys: Artist, Album, Song, Users, Playlist, Subscription and Plan. The eighth, Listening_History, is weak. The relationships between them are summarised below.

Relationship	Entities	Cardinality	Participation and attributes
Releases	Artist, Album	1:N	Album total, Artist partial
HasTrack	Album, Song	1:N	Song total, Album partial
Performs	Artist, Song	M:N	Song total (BR3), Artist partial; attribute `role`
Manages	Users, Artist	1:1	Both partial
Owns	Users, Playlist	1:N	Playlist total, Users partial
InPlaylist	Playlist, Song	M:N	Both partial; attribute `added_date`
Likes	Users, Song	M:N	Both partial; attribute `liked_date`
Subscribes	Users, Subscription	1:1	Subscription total, Users partial
OfPlan	Subscription, Plan	N:1	Subscription total, Plan partial
Listens (identifying)	Users, Song, Listening_History	1:1:N	Listening_History total, owners partial

A listening event cannot be identified on its own, since nothing about a single play is unique. It only makes sense as "this user played this song at this moment". We therefore model Listening_History as a weak entity owned by both Users and Song through the ternary identifying relationship Listens, with `listened_at` as its partial key. The same user can play the same song many times, which is why the weak entity side has cardinality N. Its full key is (`user_id`, `song_id`, `listened_at`), and an event disappears if either of its owners is deleted.

`play_count` is shown as a derived attribute because it can always be computed from Listening_History. We still store it,

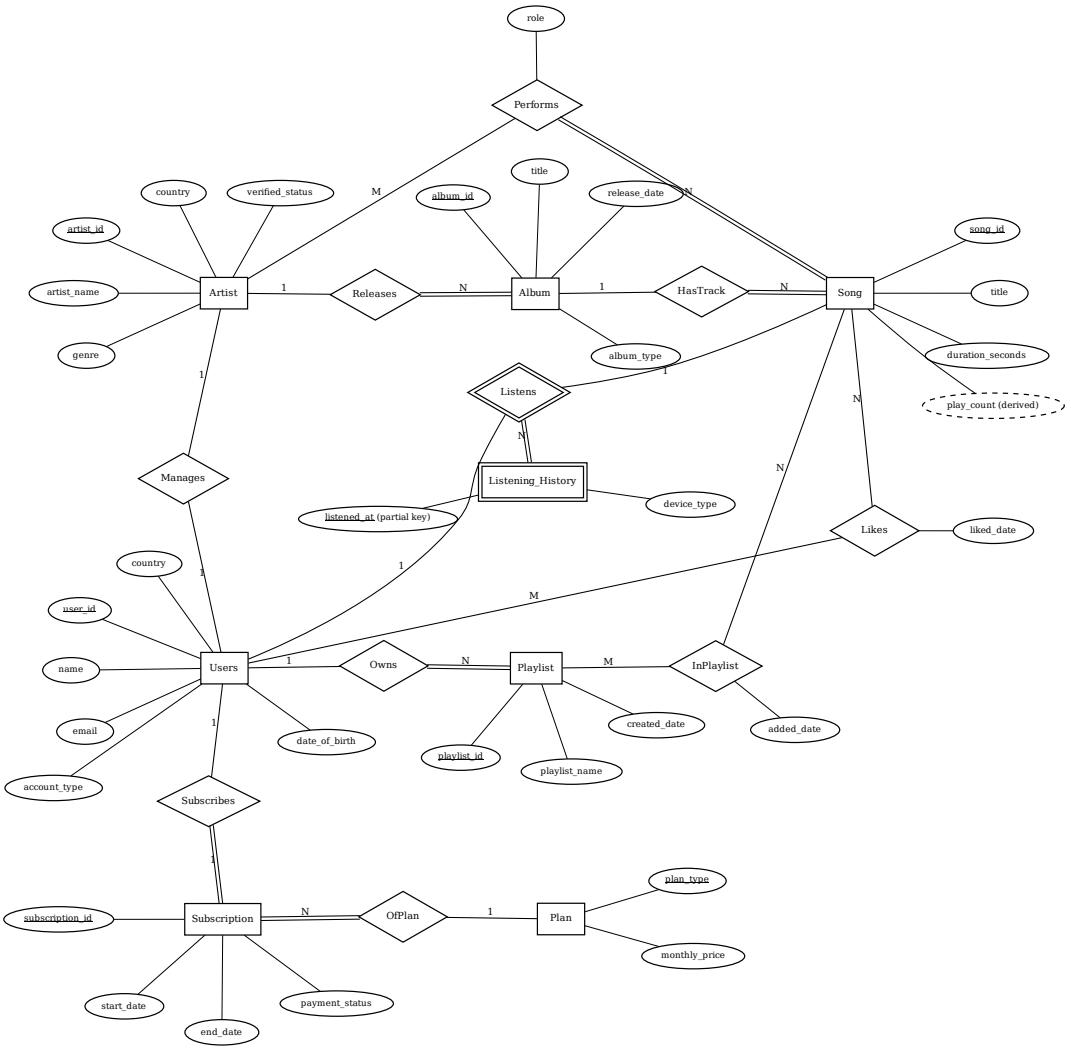


Figure 1: ER diagram in Chen notation. Double rectangles and double diamonds mark the weak entity and its identifying relationship, double lines mark total participation, underlined attributes are keys and the dashed ellipse is a derived attribute.

because charts sort by it constantly, and trigger T3 keeps it correct.

3 Relational Schema

The ER model was mapped to relations using the standard ER-to-relational mapping steps. Each strong entity became a relation with its own primary key. Listening_History, being weak, takes the keys of its two owners and adds its partial key. For every 1:N relationship the foreign key goes on the N side. The three M:N relationships (Performs, InPlaylist and Likes) each became a relation of their own, keyed by the two participating keys and holding the relationship attribute. For the two 1:1 relationships we used the foreign key approach: Subscribes puts `user_id` in SUBSCRIPTION, the side with total participation, and Manages puts `artist_id` in USERS, with a UNIQUE constraint in both cases. The result is eleven relations, shown in Figure 2 with primary keys underlined and an arrow from each foreign key to the primary key it references. `USERS.auth_id` also references the `auth.users` table that Supabase manages, which is not part of our schema and is not drawn.

The domain of each attribute and the constraints declared on it are listed below. The database is PostgreSQL hosted on Supabase, and surrogate keys use `SERIAL`.

Relation	Attribute	Domain	Constraints
ARTIST	artist_id	SERIAL	PRIMARY KEY
	artist_name	VARCHAR(150)	NOT NULL, UNIQUE (candidate key), at least 2 characters
	genre, country	VARCHAR(100)	NOT NULL
	verified_status	BOOLEAN	NOT NULL, default FALSE
ALBUM	album_id	SERIAL	PRIMARY KEY
	title	VARCHAR(200)	NOT NULL
	release_date	DATE	NOT NULL

Relation	Attribute	Domain	Constraints
	album_type	VARCHAR(10)	NOT NULL, default 'album', one of 'album', 'single', 'ep'
	artist_id	INT	NOT NULL, references ARTIST, ON DELETE RESTRICT
SONG	song_id	SERIAL	PRIMARY KEY
	title	VARCHAR(200)	NOT NULL
	duration_seconds	INT	NOT NULL, between 10 and 3600
	play_count	INT	NOT NULL, default 0, never negative (maintained by T3)
	album_id	INT	NOT NULL, references ALBUM, ON DELETE CASCADE
USERS	user_id	SERIAL	PRIMARY KEY
	auth_id	UUID	NOT NULL, UNIQUE, references auth.users, ON DELETE CASCADE
	name	VARCHAR(150)	NOT NULL, at least 2 characters
	email	VARCHAR(200)	NOT NULL, UNIQUE (candidate key), must match an email pattern
	date_of_birth	DATE	NOT NULL (minimum age checked by T4)
	country	VARCHAR(100)	NOT NULL
	account_type	VARCHAR(10)	NOT NULL, default 'listener', one of 'listener', 'artist', 'admin'
	artist_id	INT	NULL allowed, UNIQUE, references ARTIST, ON DELETE SET NULL; CHECK ((account_type = 'artist') = (artist_id IS NOT NULL))
PLAN	plan_type	VARCHAR(20)	PRIMARY KEY (values 'Premium' and 'Family')
	monthly_price	NUMERIC(6,2)	NOT NULL, greater than 0
SUBSCRIPTION	subscription_id	SERIAL	PRIMARY KEY
	user_id	INT	NOT NULL, UNIQUE (makes the relationship 1:1), references USERS, ON DELETE CASCADE
	plan_type	VARCHAR(20)	NOT NULL, references PLAN, ON DELETE RESTRICT
	start_date, end_date	DATE	NOT NULL, CHECK (end_date > start_date)
	payment_status	VARCHAR(10)	NOT NULL, default 'Pending', one of 'Paid', 'Pending', 'Failed'
PLAYLIST	playlist_id	SERIAL	PRIMARY KEY
	playlist_name	VARCHAR(200)	NOT NULL, not empty; UNIQUE together with user_id
	created_date	TIMESTAMPTZ	NOT NULL, default now()
	user_id	INT	NOT NULL, references USERS, ON DELETE CASCADE
PERFORMS	artist_id, song_id	INT	Together the PRIMARY KEY; each references its parent, ON DELETE CASCADE
	role	VARCHAR(20)	NOT NULL, either 'Lead' or 'Featured'
PLAYLIST_SONG	playlist_id, song_id	INT	Together the PRIMARY KEY; each references its parent, ON DELETE CASCADE
	added_date	TIMESTAMPTZ	NOT NULL, default now() (checked by T2)
LIKES	user_id, song_id	INT	Together the PRIMARY KEY; each references its parent, ON DELETE CASCADE
	liked_date	TIMESTAMPTZ	NOT NULL, default now()
LISTENING_ HISTORY	user_id, song_id	INT	Each references its owner, ON DELETE CASCADE
	listened_at	TIMESTAMPTZ	NOT NULL, default now(); PRIMARY KEY is (user_id, song_id, listened_at)
	device_type	VARCHAR(20)	NOT NULL, one of 'Mobile', 'Desktop', 'Web', 'Tablet', 'Smart Speaker'

4 Integrity Constraints

Entity integrity is guaranteed by the primary key on each of the eleven relations, and the UNIQUE constraints in the table above enforce the remaining candidate keys. Every foreign key states what happens when the referenced row is deleted. We cascade wherever the child has no meaning without its parent, such as a song without its album or a playlist without its owner. We restrict deletion of an artist who still has albums and of a plan that is still in use, and we set `USERS.artist_id` to NULL if an artist profile is removed, so the person's account survives. Total participation shows up as NOT NULL on the foreign keys of ALBUM, SONG, PLAYLIST and SUBSCRIPTION, and domain constraints are declared as CHECK constraints on every enumerated or bounded attribute. Row Level Security on Supabase restricts each listener to their own playlists, likes, history and subscription, and each artist to their own catalogue.

Some rules involve more than one row or relation and cannot be written as a CHECK. These are handled by the following triggers.

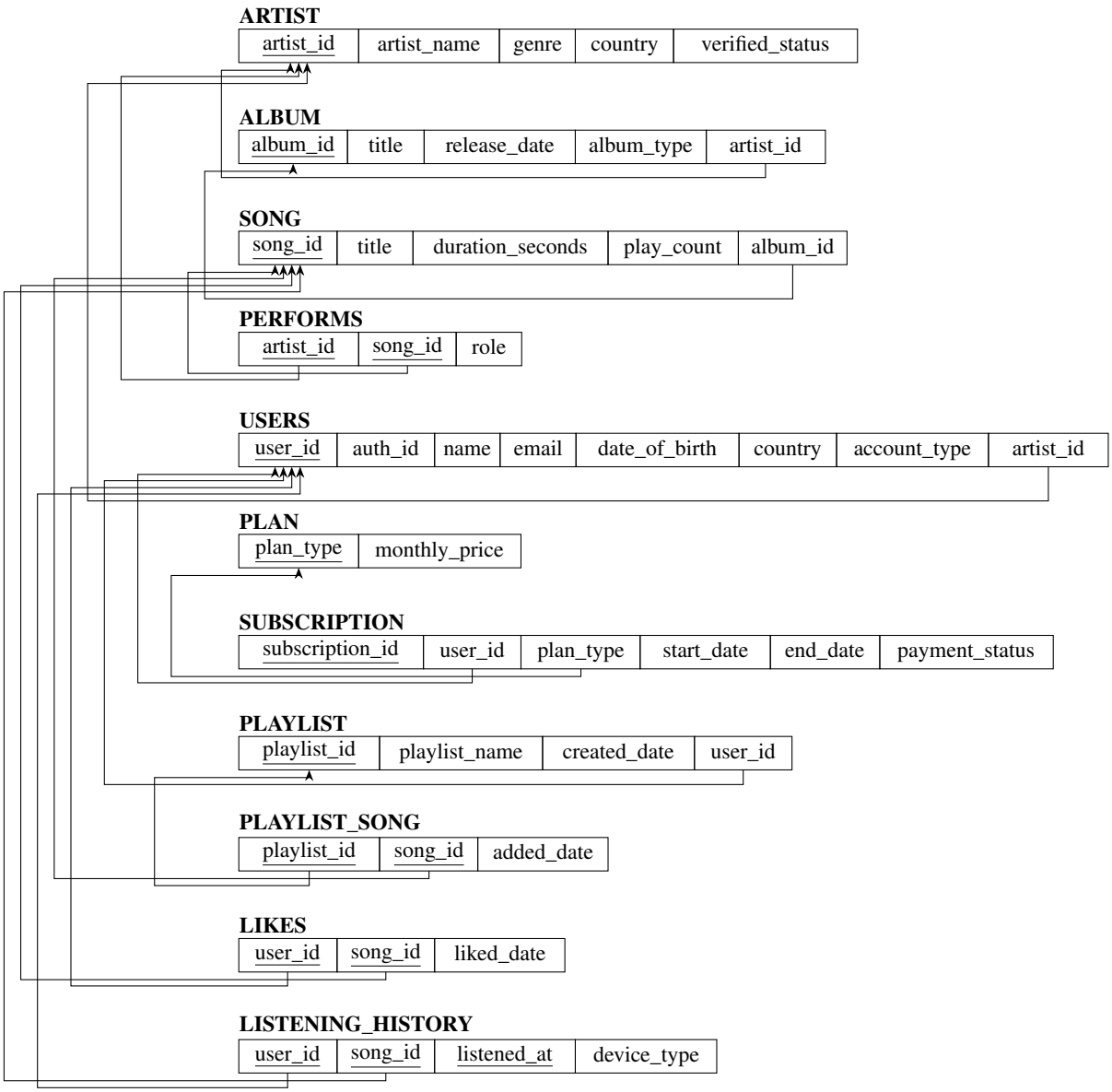


Figure 2: Relational database schema with referential integrity constraints.

Trigger	Fires on	What it does
T1 `trg_validate_listen`	LISTENING_HISTORY, before insert	Rejects a listen dated before the album's release or after the current time (BR4)
T2 `trg_validate_playlist_add`	PLAYLIST_SONG, before insert	Rejects an entry dated before the playlist's creation (BR5)
T3 `trg_play_count`	LISTENING_HISTORY, after insert or delete	Adds or subtracts one from the song's play count (BR6)
T4 `trg_validate_dates`	USERS and SUBSCRIPTION, before insert or update	Rejects users younger than 13 and subscriptions starting in the future (BR7)
T5 `trg_playlist_limit`	PLAYLIST, before insert	Rejects a sixth playlist unless the owner has a paid subscription that has not expired (BR8)

T4 is a trigger rather than a CHECK because the rule depends on today's date, and a CHECK constraint should not. BR3 is harder: it asks for total participation of Song in an M:N relationship, which a foreign key cannot express. We handle it by inserting songs only through the stored function `release_album()`, which adds the album, its songs and their performer credits in a single transaction and aborts the whole release if any song lacks a lead performer.

5 Normalization

All attributes are atomic, so every relation is in 1NF. The functional dependencies that hold are listed below. In each relation, every non-trivial dependency has a superkey on its left-hand side, which means every relation is in BCNF and therefore also in 3NF.

Relation	Functional dependencies
ARTIST	artist_id → artist_name, genre, country, verified_status artist_name → artist_id
ALBUM	album_id → title, release_date, album_type, artist_id
SONG	song_id → title, duration_seconds, play_count, album_id
USERS	user_id → every attribute email → user_id auth_id → user_id artist_id → user_id
PLAN	plan_type → monthly_price
SUBSCRIPTION	subscription_id → every attribute user_id → subscription_id
PLAYLIST	playlist_id → playlist_name, created_date, user_id {user_id, playlist_name} → playlist_id
PERFORMS	{artist_id, song_id} → role
PLAYLIST_SONG	{playlist_id, song_id} → added_date
LIKES	{user_id, song_id} → liked_date
LISTENING_HISTORY	{user_id, song_id, listened_at} → device_type

A few choices were made specifically to avoid redundancy. Songs do not carry their own release date, because it is already determined by the album. Plan prices live in a separate PLAN relation; keeping a price in SUBSCRIPTION would introduce the transitive dependency plan_type → monthly_price. An artist's monthly listener count changes every day, so it is computed by a view instead of being stored. The one deliberate exception is `play_count`. It does not break BCNF inside SONG, but it repeats information that Listening_History already holds. We accept this because chart queries read it far more often than it changes, and trigger T3 keeps the two consistent.

6 Design Decisions

Performs links artists to songs rather than to albums, because features and collaborations happen on individual tracks; the main artist of an album is already captured by Releases.

We kept the subscription in its own relation even though the relationship is 1:1. Most users are on the free tier and would otherwise carry several NULL columns in USERS, and the UNIQUE constraint on `user_id` still enforces the 1:1 limit. A renewal simply updates the existing row.

Passwords are not stored in our schema at all. Supabase Auth hashes and manages credentials in `auth.users`, and `USERS.auth_id` links each profile to its login.

Strong entities use surrogate keys, since names and titles can repeat or change over time. PLAN uses its natural key because plan names are few and stable. The associative relations use the natural composite key, which enforces BR1 without any extra constraint.

PostgreSQL does not index foreign keys automatically, so we add indexes on ALBUM(artist_id), SONG(album_id), PLAYLIST(user_id) and PERFORMS(song_id), an index on LISTENING_HISTORY(song_id, listened_at) for the chart queries, and one on lower(SONG.title) for search.

PART B: APPLICATION DESIGN

7 Technology Stack and Architecture

Layer	Technology
Frontend	React 19 with Vite 7, written in JavaScript (JSX); React Router 7, Tailwind CSS 4, Recharts and lucide-react
Data access	The supabase-js client, wrapped in one service module (`src/services/api.js`)
Backend	Supabase: Auth for sign-up and login, the PostgREST API, Row Level Security, PL/pgSQL functions called through `supabase.rpc()`, and SQL views
Database	PostgreSQL, hosted by Supabase
Version control	GitHub, with every database object kept in ordered SQL scripts

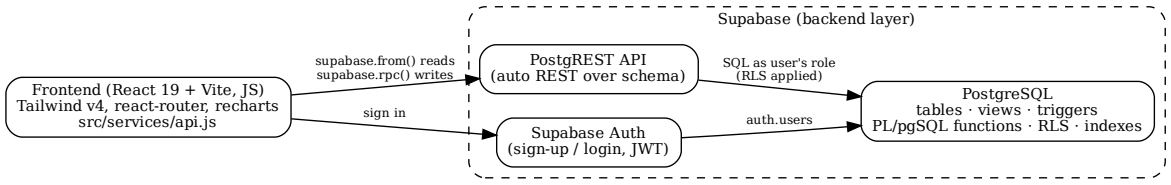


Figure 3: High-level architecture: front end, Supabase backend and PostgreSQL database.

The React front end never talks to PostgreSQL directly; everything goes through Supabase. Simple reads, such as listing an album's songs, use `supabase.from(...).select()` on a table or view, and Row Level Security filters the result to what the logged-in user may see. Anything that writes to more than one relation or has to respect a business rule is written as a PL/pgSQL function and called with `supabase.rpc()`, and each such call runs as one transaction. We plan four of these: `release_album`, `record_listen`, `add_to_playlist` and `subscribe`. All reports are computed in SQL, so the front end only displays results.

8 Application Workflows

Workflow	What happens	DBMS concepts
W1. An artist releases an album	The artist fills in the album details, the track list and the credits for each track. `release_album()` inserts rows into ALBUM, SONG and PERFORMS. If anything is wrong, say a track with an invalid duration or no lead performer, the whole release is rolled back.	Transaction, stored function, CHECK, BR3, Row Level Security
W2. A listener plays songs and checks the charts	Playing a song calls `record_listen()`, which inserts a listening event. T1 validates it and T3 updates the song's play count. The Charts and Wrapped pages then read SQL views for top songs by country, artist monthly listeners, genre trends and the listener's own summary.	Triggers, views, CTEs, window functions, aggregation, indexes
W3. A listener manages playlists and a subscription	The listener creates playlists and adds and likes songs. Adding a song that is already in the playlist fails on the primary key, and a free user's sixth playlist is rejected by T5. After subscribing through `subscribe()`, the limit no longer applies. Deleting a playlist removes its entries through ON DELETE CASCADE.	Primary and unique keys, CHECK, trigger, cascading delete, Row Level Security

9 Planned Queries and DBMS Features

#	Query	Techniques
Q1	Top 10 songs in each country over the last 30 days	CTE, joins, GROUP BY, RANK() OVER (PARTITION BY)
Q2	Monthly listeners of each artist	Joins, COUNT(DISTINCT), view
Q3	Share of listening by genre over the last 30 days	Joins, GROUP BY, SUM() OVER ()
Q4	A listener's Wrapped: top artist, top song, minutes listened, top genre	Several CTEs, aggregation
Q5	Monthly revenue by plan	Join with PLAN, aggregation, date filter
Q6	Artists with the most distinct collaborators	Self-join on PERFORMS
Q7	Songs a listener has played at least five times but not liked	GROUP BY, HAVING, NOT EXISTS
Q8	Songs played more often than the average song on their album	Correlated subquery

Beyond these queries the project uses transactions inside the four stored functions, the five triggers T1 to T5, views

(`v_artist_monthly_listeners`, `v_top_songs_by_country`, `v_genre_trends` and `v_user_wrapped`), the indexes listed in Section 6, and Row Level Security policies.

The seed scripts load about 1,250 records: artists, albums, songs, credits, users, playlists, likes, subscriptions, and around 900 listening events spread across five device types.

10 Progress and Team Contributions

So far we have completed the ER model, the mapping to the relational schema described in Part A, and the scoping of the technology stack and application architecture described in Part B.

Member	Contribution
Ritwik Sethi	Requirements analysis (problem statement, users, assumptions, business rules); ER model (entities, relationships, cardinality and participation, ER diagram); relational schema (ER-to-relational mapping, schema diagram)
Arihant Rakhecha	ER model (weak entity and identifying relationship, participation constraints); relational schema (attribute domains, keys and attribute constraints); integrity constraints (referential actions, CHECK constraints, trigger design)
Shreyas Ghosh	Normalization (functional dependencies, BCNF analysis); application design (architecture and workflows)
Shreya Agrawal	Application design (technology stack, planned queries and DBMS features); documentation (design report)
Piyush Gaikwad	Normalization (redundancy analysis and the treatment of derived attributes); documentation (README and GitHub repository)

GitHub repository: [link to be added]